SUMMIT BOOTCAMP · PROGRAMMING AND DEVELOPMENT TRACK

QuoteIt.

Software Requirements Specification

A community vault for Arabic cybersecurity quotes — browse the collection, share a lesson, and like the voices that matter.

Document version	2.0 (final)
Supersedes	Version 1.2 — October 2026
Project type	Individual final project
Programme	Summit Bootcamp — Programming and Development Track
Date	27 October 2026
Implementation	Laravel 13.37 · PHP 8.5 · Blade · Tailwind CSS 4 · SQLite
Document status	Final — verified against the implemented application

> *This revision replaces the earlier draft and describes QuoteIt as it is actually built: a database-backed Arabic quote feed with a dark responsive interface, owner-only quote controls, and a persistent Like feature.*

Revision History

This document has been revised so that every statement matches the final implemented application. The earlier version described the planned interface and did not include the Like feature or the Arabic-first presentation.

Table 1. Revision history

Version	Date	Summary of changes
1.0	**October 2026**	**Initial specification of the QuoteIt quote-sharing platform.**
1.2	**October 2026**	**Refined scope, functional requirements, and use case diagram; still based on the planned interface.**
2.0	**27 October 2026**	**Full revision against the final implementation: Summit Bootcamp context; Laravel 13 / PHP 8.5 stack; database-backed Arabic cybersecurity quotes; RTL typography; dark responsive redesign; guest login prompt; owner-only controls; and the Like/Unlike feature. Adds acceptance criteria, non-functional requirements, data model, traceability, and a demo appendix.**

Scope of this revision:

- Replaces the generic quote-sharing description with the cybersecurity community vault that was actually built.
- Aligns every flow, field rule, and character limit with the final source code and seeded database.
- Adds Like/Unlike and redraws the use case diagram to include it.
- Corrects statements that did not match the final interface, including the claim that every field on the login page shows a hint.
- Updates the version, date, programme, technology stack, and verification evidence.

Table of Contents

1. Introduction **4**
1.1 Purpose 4
1.2 Scope 4
1.3 Intended Audience and Reading Guide 4
1.4 Definitions and Acronyms 4
1.5 References 5
2. Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 6
2.3 User Classes and Characteristics 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 Assumptions and Dependencies 7
3. System Features **7**
3.1 View Quotes (FR-01) 7
3.2 Create Account (FR-02) 7
3.3 Log In (FR-03) 8
3.4 Log Out (FR-04) 8
3.5 Share a Quote (FR-05) 8
3.6 Edit Own Quote (FR-06) 9
3.7 Delete Own Quote (FR-07) 9
3.8 Like and Unlike a Quote (FR-08, New Feature) 9
4. Use Case Model **11**
4.1 Actors and System Boundary 11
4.2 Use Case Summary 11
4.3 Relationships and Constraints 12
5. External Interface Requirements **13**
5.1 User Interface Overview 13
5.2 Pages and Routes 13
5.3 Forms, Validation, and Feedback 14
5.4 Software Interfaces 15
5.5 Communication Interfaces 15
6. Non-Functional Requirements **16**
7. Data Requirements **18**
7.1 Data Model 18
7.2 users 18
7.3 quotes 18
7.4 quote_likes 19
7.5 Relationships and Integrity Rules 19
7.6 Seed Data and Demonstrated Content 19
7.7 Data Retention and Operational Notes 20
8. Requirements Traceability **20**
9. Changes from the Original SRS **21**
10. Future Enhancements **21**
11. Appendix A — Setup and Demonstration **22**
11.1 Prerequisites 22
11.2 Installation and Run 22
11.3 Demo Account 22
11.4 Suggested Five-Minute Demo Flow 22
11.5 Automated Check 23
Appendix B — Verification Checklist **24**

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements of QuoteIt, the final project submitted for the Summit Bootcamp, Programming and Development Track. It describes the system that was implemented, the behaviour a user can rely on, the data that is stored, and the constraints under which the application operates.

The document is also the reference for evaluation and demonstration. It deliberately describes only capabilities that exist in the final build; ideas that were considered but not implemented are listed separately in Section 10.

1.2 Scope

QuoteIt is a server-rendered Laravel web application that publishes a curated collection of short Arabic cybersecurity quotes. Anyone may read the collection. A registered user may create an account, sign in, publish a quote of 3–155 characters, edit or delete their own quotes, like or unlike any quote, and sign out. All content is stored in SQLite and rendered through Blade templates styled with Tailwind CSS.

The final application is Arabic-first: Arabic quote text and author names are detected automatically and displayed right-to-left with the IBM Plex Sans Arabic typeface, while the surrounding interface remains in English.

Table 2. Scope of the final system

In scope	Out of scope
Public quote feed with author, date, and like count • Account creation, login, logout, and remember-me • Publishing quotes of 3–155 characters • Owner-only edit and delete • Persistent like/unlike for signed-in users • Arabic or English input with automatic RTL rendering • Responsive dark interface from 320 px upward • Server-side validation and success or error feedback • Seeded demo accounts and eight Arabic quotes	**Comments, replies, or following • Categories, tags, search, or filters • Public user profiles • Notifications or email • Moderation, reporting, or an admin dashboard • Password reset or email verification • Image or file uploads • Public API or native mobile app • Pagination or infinite scrolling**

The mandatory new feature for the final project is Like/Unlike. It is implemented end to end: interface, route, controller action, database table, unique constraint, and persistent count.

1.3 Intended Audience and Reading Guide

The document is written for three readers. Each can focus on the sections that matter most.

Table 3. Intended audience

Reader	Recommended sections
Evaluator or instructor	**Sections 1, 3, 4, 8, and 9, plus Appendix B for the verification checklist.**
Developer or maintainer	**Sections 2, 3, 6, and 7, and Appendix A for setup and project operation.**
Demo presenter	**Section 3 for behaviour, Section 4 for the use case model, and Appendix A for the suggested demo flow.**

1.4 Definitions and Acronyms

Table 4. Definitions used in this document

Term	Definition
Guest	**An unauthenticated visitor who can read the public quote feed.**

Term	Definition
Registered user	An account holder with an active session who can publish and manage quotes.
Owner	The registered user who created a particular quote; only the owner may edit or delete it.
Quote	A short text of 3–155 characters published by a registered user.
Like	A persistent association between one user and one quote. A user may like a quote at most once.
Like count	The number of distinct users who currently like a quote, computed from the database.
RTL	Right-to-left writing direction, used for Arabic quote text and Arabic author names.
Seed data	Records inserted by the database seeder so the application can be demonstrated immediately.
Blade	Laravel's server-side template engine, used with inheritance and reusable components.
Eloquent	Laravel's object-relational mapper, used for the User, Quote, and like relationships.
CSRF	Cross-Site Request Forgery protection; every state-changing form carries a token.
Vite	The front-end build tool that compiles the Tailwind CSS and JavaScript assets.

1.5 References

- Summit Bootcamp — Final Project Brief, Programming and Development Track, 2026.
- QuoteIt application source code, routes, migrations, seeders, and README (project repository).
- Laravel 13 documentation: routing, controllers, validation, Eloquent, authentication, sessions, and Blade.
- Tailwind CSS 4 documentation and Vite 8 documentation.
- PHP 8.5 manual: PDO SQLite, password hashing, and session handling.
- SQLite documentation: foreign keys, unique constraints, and cascade actions.

2. Overall Description

2.1 Product Perspective

QuoteIt is a standalone Laravel application that follows the Model–View–Controller pattern. It does not depend on an external API, a third-party database service, or a JavaScript framework. The browser sends standard HTTP requests; Laravel routes them to controllers; controllers validate input and call Eloquent models; Blade renders the HTML; SQLite persists the data. Vite compiles the Tailwind stylesheet and the application's JavaScript entry point, which currently contains no custom code beyond the browser's native confirmation dialog.

Table 5. Application layers

Layer	Responsibility	Implementation
Presentation	Pages, cards, forms, and feedback	Blade templates with inheritance and reusable components; Tailwind CSS 4
Application	Routing, validation, authentication, authorization	Laravel routes, controllers, auth middleware, and server-side ownership checks
Domain	Entities and relationships	Eloquent models User and Quote; belongsTo and belongsToMany relations
Data	Persistence and integrity	SQLite database created by migrations and populated by a seeder
Build	Asset compilation and fonts	Vite 8 with the Laravel Vite plugin; Google Fonts for Space Grotesk and IBM Plex Sans Arabic

2.2 Product Functions

The final system provides the following user-facing functions:

- Browse a public, newest-first quote feed showing text, author, date, and like count.
- Create an account with a validated name, unique email address, and confirmed password.
- Log in with optional remember-me and log out securely.
- Publish a quote of 3–155 characters in Arabic or English.
- Edit or delete only the quotes owned by the signed-in user.
- Like or unlike any quote and see the count stored in the database.
- Read Arabic quotes and author names in correct right-to-left layout and typography.
- Use the same interface on phones, tablets, and desktop screens.

2.3 User Classes and Characteristics

Table 6. User classes

User class	Authentication	Capabilities	Technical skill
Guest	**None**	**Read all quotes, see like counts, open sign-up and login pages, read the guest sharing prompt.**	**Basic browser use**
Registered user	**Email and password**	**Everything a guest can do, plus publish quotes, like or unlike quotes, and log out.**	**Basic browser use**
Owner	**Registered user who created the quote**	**Edit or delete that quote and see the "Your quote" label. Ownership is checked on the server.**	**Basic browser use**

2.4 Operating Environment

Table 7. Operating environment

Component	Requirement
Server runtime	**PHP 8.5.0 (the Composer constraint accepts PHP 8.3 or newer).**
Framework	**Laravel 13.37.**
Database	**SQLite 3 through Laravel's PDO driver.**
View layer	**Blade templates with inheritance and components.**
Styling	**Tailwind CSS 4 compiled by Vite 8.**
Client	**A current version of Chrome, Edge, Firefox, or Safari with JavaScript enabled for the delete confirmation.**
Screen size	**320 px and wider; the feed uses one, two, or three columns depending on width.**
Host	**Windows, macOS, or Linux with PHP, Composer, Node.js, and npm available.**
Fonts	**Google Fonts (Space Grotesk and IBM Plex Sans Arabic) with system sans-serif fallbacks.**

2.5 Design and Implementation Constraints

- The application must follow Laravel conventions: named routes, thin controllers, Eloquent models, migrations, and a seeder.
- All pages must use Blade template inheritance and reusable components rather than duplicated markup.
- Quote content is limited to 155 characters in both the database column and the validation rule; the minimum is 3 characters.
- Only the owner may edit or delete a quote; the rule must be enforced on the server, not only by hiding buttons.
- Passwords must be hashed with Laravel's hashed cast and must never be stored or displayed in plain text.

- Every state-changing form must include a CSRF token.
- Arabic content must be rendered with dir="rtl", lang="ar", right alignment, and the Arabic typeface.
- The interface must remain usable without a JavaScript framework; the only custom browser behaviour is the native delete confirmation.
- The feed is rendered as a single list; pagination is intentionally omitted because the seeded collection is small.
- Guests may read like counts but may not change them; liking requires an authenticated session.

2.6 Assumptions and Dependencies

- Users have a modern browser and, for the intended typography, access to Google Fonts; otherwise the CSS fallback fonts are used.
- SQLite is sufficient for a single-instance, low-traffic demonstration and evaluation environment.
- The repository includes a seeded SQLite database, so the demo account and eight quotes are available immediately.
- Email addresses are unique but are not verified; the application sends no email.
- The application is not designed for high-concurrency write load or horizontal scaling.
- Timestamps are stored in the application timezone and displayed as abbreviated English dates (for example, Mar 5, 2026).

3. System Features

The eight functional requirements below define the complete behaviour of the final system. Each requirement states the actors, preconditions, main flow, alternative flows, postconditions, and acceptance criteria. The identifiers FR-01 to FR-08 are used again in the traceability matrix in Section 8.

3.1 View Quotes (FR-01)

> **FR-01 —** The system shall display every published quote to every visitor, ordered newest first, with the quote text, author name, publication date, and current like count.

Actors: Guest and registered user.

Preconditions: None; the quote feed is public.

Main flow: The visitor opens the home page → the controller loads every quote with its author and aggregate like count, newest first → the page renders the hero statistics and the quote grid → each card shows the quote, author, date, like count, and only the controls allowed for that visitor → Arabic quote text and Arabic author names are rendered right-to-left with the Arabic typeface.

Alternative flows: If the collection is empty, an empty-state card explains that no quotes exist yet. A guest who attempts a member action is sent to the login page.

Postconditions: The collection is displayed; no data is changed.

Acceptance criteria: A guest sees all seeded quotes from SQLite in newest-first order. Arabic content is right-aligned. The layout has no horizontal overflow at 320, 390, 768, 1024, and 1440 px.

3.2 Create Account (FR-02)

> **FR-02 —** The system shall allow a visitor to create an account with a name of 2–50 characters, a valid unique email address, and a password of at least 8 characters confirmed by re-entry.

Actors: Guest.

Preconditions: The visitor is not authenticated.

Main flow: The visitor opens the sign-up page → enters full name, email, password, and password confirmation → submits the form → the system validates every field → the password is hashed by the User model cast → the account is created → the user is logged in and the session is regenerated → the browser is redirected to the home page.

Alternative flows: Name shorter than 2 or longer than 50 characters → inline error. Invalid or already registered email → inline error. Password shorter than 8 characters or confirmation mismatch → inline error. In every case the account is not created and password fields are cleared.

Postconditions: A user row exists with a hashed password; an authenticated session is active.

Acceptance criteria: Valid data creates the account and signs the user in. A duplicate email is rejected. The stored password is a hash, never the plain value.

3.3 Log In (FR-03)

> **FR-03 —** The system shall authenticate a registered user with an email address and password and shall optionally keep the session active on the device.

Actors: Guest or signed-out registered user.

Preconditions: An account exists and the visitor is not authenticated.

Main flow: The user opens the login page → enters email and password, optionally selects Remember me → submits → the system validates the required fields → Laravel attempts authentication against the stored hash → the session is regenerated → the user is redirected to the originally requested page or the home page.

Alternative flows: Credentials do not match → a general error is shown, the email is preserved, and no session is created. Missing or malformed fields → inline errors.

Postconditions: An authenticated session exists; the header shows the user's name and a Log out button.

Acceptance criteria: The demo credentials sign in successfully. A wrong password creates no session. Remember me issues a persistent login cookie.

3.4 Log Out (FR-04)

> **FR-04 —** The system shall allow an authenticated user to end the session.

Actors: Registered user.

Preconditions: The user is authenticated.

Main flow: The user selects Log out in the header → the browser posts the logout form with a CSRF token → the system logs the user out, invalidates the session, and regenerates the CSRF token → the user is redirected to the home page as a guest.

Alternative flows: A guest who posts to the logout route is returned to the home page without an error.

Postconditions: No authenticated session remains; protected routes redirect to the login page.

Acceptance criteria: After logout the header shows Log in and Sign up, and posting to a protected route redirects to the login page.

3.5 Share a Quote (FR-05)

> **FR-05 —** The system shall allow an authenticated user to publish a quote of 3–155 characters and shall attribute it to that user.

Actors: Registered user.

Preconditions: The user is authenticated.

Main flow: The signed-in user sees the share form at the top of the home page → writes a quote in Arabic or English → submits POST /quotes with a CSRF token → the system validates the content as required, string, minimum 3, and maximum 155 → the quote is stored with the current user's ID and timestamps → the user is redirected to the home page with a success alert → the new quote appears first in the feed.

Alternative flows: Empty, whitespace-only, shorter than 3, or longer than 155 characters → the quote is not stored; an error alert and an inline field error are shown. A guest sees the login prompt instead of the form; a guest POST is redirected to login.

Postconditions: A quote row exists, owned by the current user and visible to every visitor.

Acceptance criteria: 3 and 155 characters are accepted; 2 and 156 are rejected. The author is the signed-in user. Arabic text is accepted and displayed RTL. A success alert confirms publication.

3.6 Edit Own Quote (FR-06)

> **FR-06 —** The system shall allow a user to edit only the quotes they created.

Actors: Registered user in the owner role.

Preconditions: The user is authenticated and owns the quote.

Main flow: The owner selects Edit on their quote → the edit page loads with the current text → the owner changes the text and submits PUT /quotes/{quote} → the system verifies ownership → validates 3–155 characters → updates the content and updated_at timestamp → redirects with a success alert.

Alternative flows: A different signed-in user requests the edit URL → HTTP 403 and no form is shown. A guest → redirected to login. Invalid length → inline error and no update.

Postconditions: The quote text is updated; its author, creation date, and likes are preserved.

Acceptance criteria: The owner can update the quote; another user receives HTTP 403; the length rule is enforced on update as well as on create.

3.7 Delete Own Quote (FR-07)

> **FR-07 —** The system shall allow a user to delete only the quotes they created.

Actors: Registered user in the owner role.

Preconditions: The user is authenticated and owns the quote.

Main flow: The owner selects Delete → the browser asks for confirmation → DELETE /quotes/{quote} is posted with a CSRF token and method override → the system verifies ownership → deletes the quote → cascades deletion of its like rows → redirects with a success alert → the feed and counters update.

Alternative flows: A non-owner attempts deletion → HTTP 403 and the quote remains. A guest → redirected to login. Cancelling the confirmation dialog does nothing.

Postconditions: The quote and all of its likes are removed.

Acceptance criteria: Owner deletion removes one quote and its likes; another user cannot delete it; the quote count and like total decrease accordingly.

3.8 Like and Unlike a Quote (FR-08, New Feature)

> **FR-08 —** The system shall allow an authenticated user to like or unlike any quote and shall display the stored like count to every visitor.

Actors: Registered user; guests observe the count.

Preconditions: The user is authenticated and the quote exists.

Main flow: The user presses the heart button on a quote → POST /quotes/{quote}/like with a CSRF token → the system toggles the user's association in quote_likes → the browser returns to the feed → the heart becomes filled or unfilled and the count changes by one.

Alternative flows: A guest sees the count in a non-interactive control labelled “Log in to like quotes”; a guest POST is redirected to login. Pressing the button again removes the existing like.

Postconditions: The association is added or removed; the count is derived from the table.

Acceptance criteria: Liking increments the count by one and unliking decrements it by one; the state persists after reload; a second user adds another like; guests cannot change the count.

Business rules:

- At most one like per user per quote, enforced by UNIQUE(quote_id, user_id).
- Like counts are computed from quote_likes; they are not stored as a column on quotes.

- Deleting a quote or a user cascades to the related like rows.
- A user may like their own quote.
- Liking produces no notification, email, or activity entry.

4. Use Case Model

The use case model describes what each actor can accomplish with QuoteIt. It contains two human actors — Guest and Registered user — and eight use cases inside the system boundary. Owner is a role played by a registered user rather than a separate actor; ownership is enforced by the server whenever a quote is edited or deleted.

4.1 Actors and System Boundary

- Guest: an unauthenticated visitor who can view quotes, create an account, and log in.
- Registered user: an authenticated account holder who can view quotes, publish a quote, like or unlike any quote, manage their own quotes, and log out.
- Owner: the registered user who created a specific quote. The role grants edit and delete rights for that quote only.

The diagram below shows the eight use cases and their direct associations. The Like/Unlike use case is the new feature added in the final implementation; it appears alongside the original quote-management use cases.

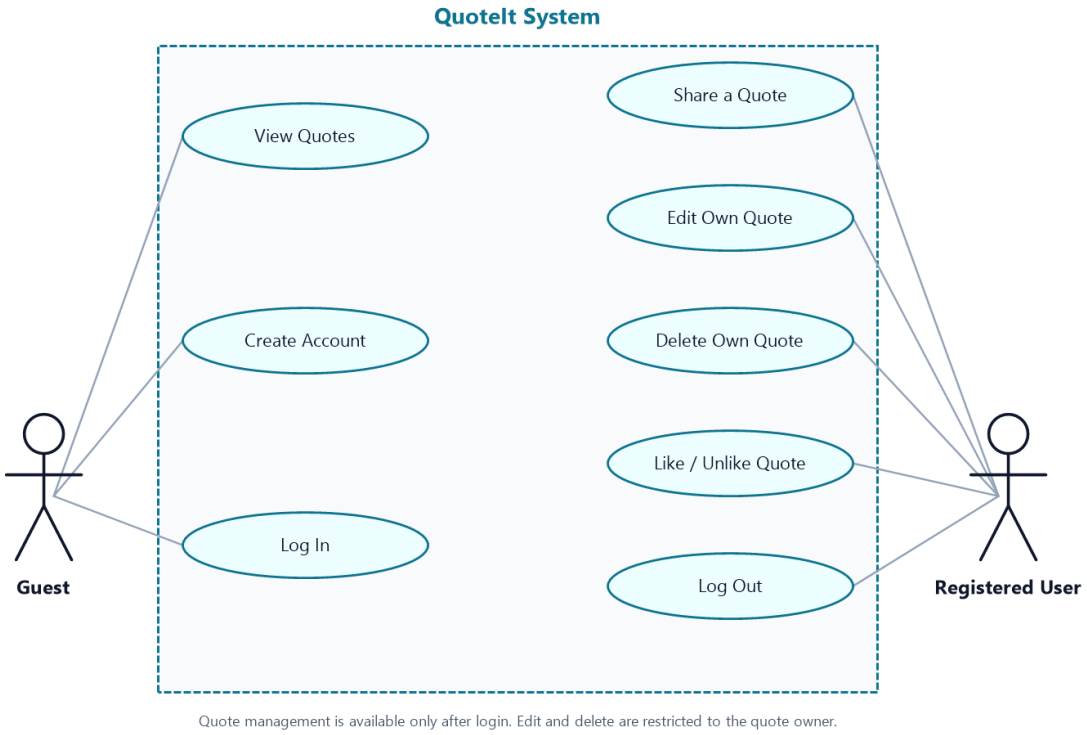


Figure 1. Use case diagram of the final QuoteIt system, including Like/Unlike.

4.2 Use Case Summary

Table 8. Use case summary

ID	Use case	Primary actor(s)	Goal
UC-01	View quotes	Guest, Registered user	Read every quote with its author, date, and like count, newest first.
UC-02	Create account	Guest	Register with a validated name, unique email, and confirmed password, then be signed in.
UC-03	Log in	Guest, Registered user	Start an authenticated session with email and password.

ID	Use case	Primary actor(s)	Goal
UC-04	**Log out**	**Registered user**	**End the session and invalidate its data.**
UC-05	**Share a quote**	**Registered user**	**Publish a quote of 3–155 characters under the current account.**
UC-06	**Edit own quote**	**Owner**	**Change the text of a quote the user created.**
UC-07	**Delete own quote**	**Owner**	**Remove a quote the user created and cascade its likes.**
UC-08	**Like / Unlike quote**	**Registered user**	**Toggle a persistent like and update the displayed count.**

4.3 Relationships and Constraints

The diagram uses direct associations rather than actor generalization so that guest-facing and member-facing actions are easy to distinguish. A registered user also has every read capability of a guest; create account is meaningful only while signed out, and log in becomes available again after logging out.

No include or extend relationships are necessary: each flow is short and self-contained. Two constraints are critical to correctness. First, edit and delete are limited to the owner and are rejected with HTTP 403 for any other signed-in user. Second, Like/Unlike requires an authenticated session; guests see the count but cannot change it. The new feature is available for every quote, including quotes written by the person pressing the button.

5. External Interface Requirements

5.1 User Interface Overview

The interface uses a dark slate canvas with cyan and blue radial accents, translucent cards, rounded corners, and a shield-shaped brand mark. Cyan marks primary actions and active states; rose marks the liked state and destructive actions. The quote feed uses one column on phones, two on medium screens, and three on wide screens.

A sticky header contains the brand, a Quotes link, and either Log in and Sign up or the signed-in user's name and Log out. The home page opens with a hero section and live statistics for quotes, voices, and likes. It then shows the sharing area — a form for members or a login prompt for guests — followed by the collection.

Arabic content is detected with a Unicode Arabic range. Matching quotes and author names receive dir=rtl, lang=ar, right alignment, and the IBM Plex Sans Arabic typeface; Latin text uses Space Grotesk. The same rule applies to the author name shown in the header and in the share form.

Accessibility support includes a skip-to-content link, semantic header, main and footer landmarks, associated form labels, aria-pressed on the like button, descriptive aria-labels and titles on icon-only controls, visible focus styles, and reduced-motion handling for card hover effects.

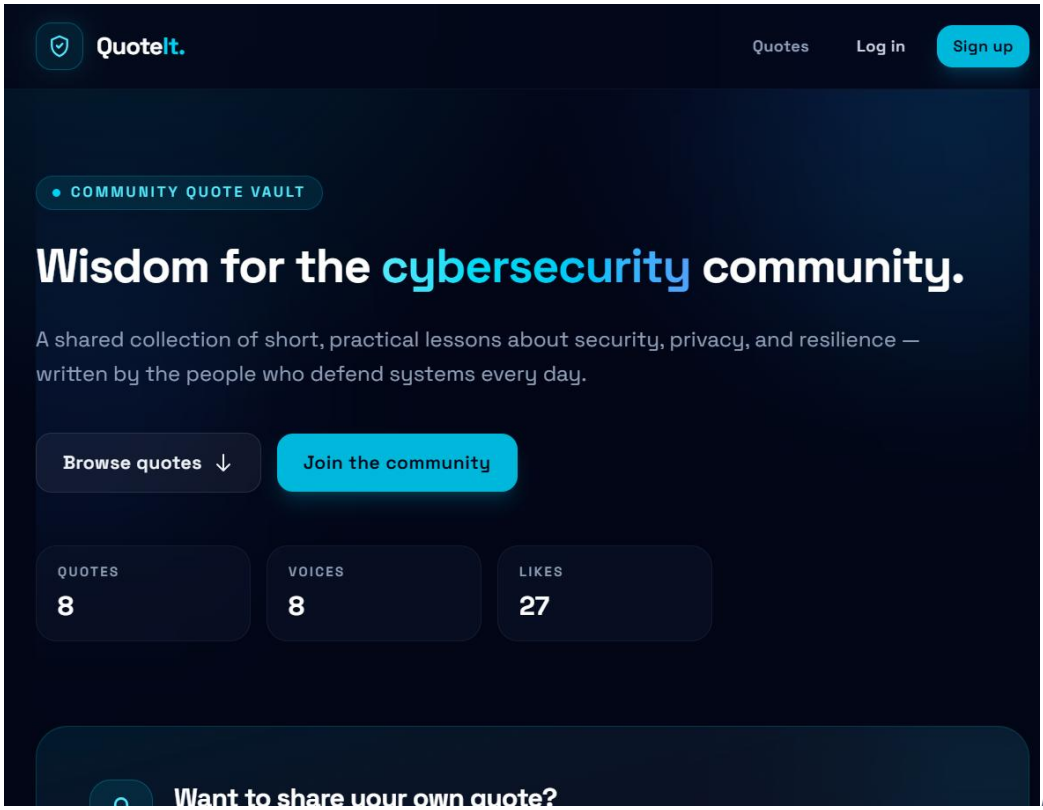


Figure 2. Final home page in guest view. The statistics are computed from SQLite; the sharing area becomes a form after login.

5.2 Pages and Routes

Table 9. Pages and routes of the final system

Page or action	Method and route	Access	Purpose
Quotes list	**GET /**	**Public**	**Hero statistics, share area or guest prompt, and the full quote feed.**

Page or action	Method and route	Access	Purpose
Sign up	GET /signup, POST /signup	Guest	Create an account and start a session.
Log in	GET /login, POST /login	Guest	Authenticate and optionally stay signed in.
Log out	POST /logout	Authenticated	Invalidate the session and return to the guest view.
Share quote	POST /quotes	Authenticated	Validate and store a quote of 3–155 characters.
Edit form	GET /quotes/{quote}/edit	Owner	Load the current quote text for editing.
Update quote	PUT /quotes/{quote}	Owner	Validate and save changes to an owned quote.
Delete quote	DELETE /quotes/{quote}	Owner	Delete an owned quote and cascade its likes.
Like toggle	POST /quotes/{quote}/like	Authenticated	Add or remove the current user's like.

5.3 Forms, Validation, and Feedback

Validation is performed on the server for every form. The table below records the rules and the guidance a user actually sees in the final interface.

Table 10. Form fields, rules, and on-screen guidance

Form	Field	Rule	Guidance shown
Sign up	Full name	Required, 2–50 characters	"2-50 characters."
Sign up	Email	Required, valid email, unique, maximum 255	"Use a valid email address you can access."
Sign up	Password	Required, minimum 8 characters	"At least 8 characters."
Sign up	Confirm password	Must match the password	"Repeat the password exactly."
Log in	Email	Required, valid email	Placeholder operator@quoteit.com; demo-account panel below the form.
Log in	Password	Required	Placeholder dots; demo-account panel below the form.
Log in	Remember me	Optional checkbox	"Remember me on this device."
Share quote	Quote	Required, 3–155 characters	"Arabic or English — 3-155 characters." plus a three-point "Before you post" panel.
Edit quote	Quote	Required, 3–155 characters	"3-155 characters."

Feedback follows the same pattern throughout the application:

- A green success alert confirms a published, updated, or deleted quote.
- A red alert at the top of the page and an inline message beside the field report quote validation errors.
- Sign-up and login failures show a general alert plus inline field errors; the login form preserves the email address.
- Deleting a quote asks for confirmation through the browser's native dialog before the request is sent.
- The guest prompt offers Log in and Create account, and the guest like control is labelled "Log in to like quotes".

The login page deliberately uses placeholders and a demo-account panel instead of per-field hints. The sign-up and quote forms show explicit hints. This document describes the interface as built rather than the earlier draft, which claimed that every field displayed a hint.

5.4 Software Interfaces

- Database: SQLite through Laravel's PDO driver, accessed with the Eloquent models User and Quote and the quote_likes pivot table.
- Templates: Blade with layout inheritance and reusable components for the brand, buttons, text inputs, text areas, form hints, input errors, and alerts.
- Assets: Vite 8 compiles Tailwind CSS 4 and the JavaScript entry point. There is no JavaScript framework; the entry point currently contains no custom code.
- Fonts: Space Grotesk and IBM Plex Sans Arabic are requested from Google Fonts over HTTPS, with system sans-serif fallbacks.
- Sessions and protection: sessions are stored in the database, and Laravel's CSRF middleware protects every POST, PUT, and DELETE form.

5.5 Communication Interfaces

- The client and server communicate over HTTP or HTTPS using standard HTML forms; there is no JSON API.
- After a successful mutation the server redirects back to the feed (POST–Redirect–GET) and flashes a success message through the session.
- Session and remember-me cookies are managed by Laravel; no third-party analytics, advertising, or tracking scripts are loaded.
- The only external network requests are for the Google Fonts stylesheet and font files; if they fail, the local fallback fonts are used.

6. Non-Functional Requirements

The non-functional requirements define the quality attributes of the final system. Each requirement is mapped to a concrete implementation or to evidence collected during verification.

Table 11. Non-functional requirements

ID	Category	Requirement	Implementation and evidence
NFR-01	Security — passwords	Passwords must be stored only as hashes and never displayed.	The User model uses Laravel's hashed cast (bcrypt). Password fields are never re-populated, and the database stores only the hash.
NFR-02	Security — authorization	Only the owner may edit or delete a quote; guests may not write.	Authenticated routes plus an ownership check that aborts with HTTP 403. The interface also hides owner controls from other users.
NFR-03	Security — sessions	State-changing requests must be protected and sessions must be refreshed.	CSRF tokens on every form; session regeneration on login; session invalidation and token regeneration on logout.
NFR-04	Security — validation	All user input must be validated on the server.	Laravel validation rules cover name, email, password confirmation, and quote length; errors are shown inline.
NFR-05	Privacy	Collect only the data needed and never expose email addresses publicly.	The users table stores name, email, and password. The feed exposes the author's name only. No analytics or tracking is present.
NFR-06	Usability	A new user must be able to publish a first quote in under one minute.	The single-field share form sits at the top of the home page after login, with a hint and a success alert.
NFR-07	Accessibility	The interface must be usable with a keyboard and assistive technology.	Skip link, semantic landmarks, associated labels, aria-pressed, descriptive icon labels, focus styles, and reduced-motion support.
NFR-08	Responsiveness	The interface must work from 320 px to 1440 px without horizontal scrolling.	Tailwind breakpoints switch the feed between one, two, and three columns; verified at 320, 390, 768, 1024, and 1440 px.
NFR-09	Localization	Arabic content must render right-to-left with appropriate typography; English must remain left-to-right.	Unicode Arabic detection, dir and lang attributes, right alignment, IBM Plex Sans Arabic, and dir=auto on name inputs.
NFR-10	Performance	The quote page must load within 3 seconds.	Quotes are eager-loaded with an aggregate like count, assets are compiled, and the seeded page measured well under one second locally.
NFR-11	Reliability and integrity	Likes must be unique and must not outlive their quote or user.	UNIQUE(quote_id, user_id); foreign keys with cascade delete; like counts are derived rather than stored.
NFR-12	Maintainability	The code must follow framework conventions and be easy to extend.	MVC separation, named routes, migrations, a seeder, reusable Blade components, and a Pest test suite.
NFR-13	Portability	The application must run on a standard PHP and Laravel environment.	Composer and npm dependencies; environment-driven database configuration; a portable schema.

ID	Category	Requirement	Implementation and evidence
NFR-14	**Compatibility**	**The application must work in current major browsers.**	**Standard HTML, CSS, and form behaviour; no proprietary browser APIs; the only scripted interaction is the native confirmation dialog.**

7. Data Requirements

7.1 Data Model

QuoteIt uses three domain tables. A user writes many quotes; a user likes many quotes and a quote is liked by many users through the quote_likes join table. Both foreign keys in the join table use cascade delete, and a unique constraint prevents duplicate likes. The entity relationship diagram below shows the final schema.

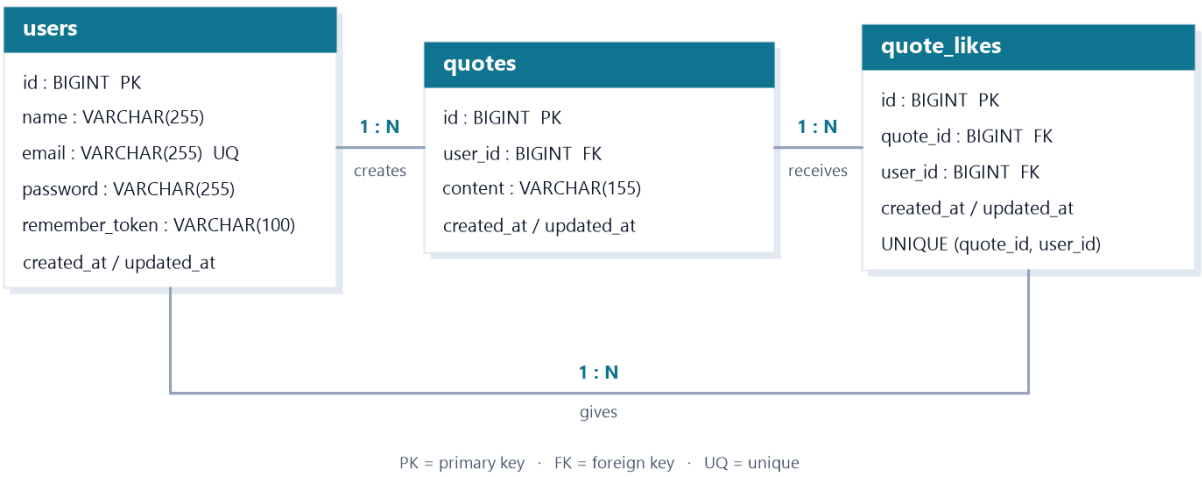


Figure 3. Entity relationship diagram of the final QuoteIt database.

The database also contains Laravel's framework tables for sessions, cache, jobs, and password-reset tokens. They support the runtime and are not part of the application's domain model; no password-reset screen is exposed.

7.2 users

Table 12. users table

Column	Type	Constraints
id	BIGINT UNSIGNED	Primary key, auto-increment.
name	VARCHAR(255)	Required; validated to 2–50 characters.
email	VARCHAR(255)	Required and unique; validated as an email address.
email_verified_at	TIMESTAMP	Nullable. Present but unused because email verification is not implemented.
password	VARCHAR(255)	Required; stored as a bcrypt hash.
remember_token	VARCHAR(100)	Nullable; supports the remember-me option.
created_at, updated_at	TIMESTAMP	Nullable; maintained by Eloquent.

7.3 quotes

Table 13. quotes table

Column	Type	Constraints
id	BIGINT UNSIGNED	Primary key, auto-increment.
user_id	BIGINT UNSIGNED	Foreign key to users.id; required; cascade delete.
content	VARCHAR(155)	Required; validated to 3–155 characters.

Column	Type	Constraints
created_at	TIMESTAMP	Nullable; set on creation and used for newest-first ordering and the displayed date.
updated_at	TIMESTAMP	Nullable; changed whenever the quote is edited.

7.4 quote_likes

Table 14. quote_likes table

Column	Type	Constraints
id	BIGINT UNSIGNED	Primary key, auto-increment.
quote_id	BIGINT UNSIGNED	Foreign key to quotes.id; required; cascade delete.
user_id	BIGINT UNSIGNED	Foreign key to users.id; required; cascade delete.
created_at, updated_at	TIMESTAMP	Nullable; maintained by Eloquent.

Unique constraint: (quote_id, user_id). A user can like a given quote at most once; pressing the heart again removes the row.

7.5 Relationships and Integrity Rules

- User 1:N Quote — each quote belongs to exactly one author; deleting a user deletes that user's quotes.
- User M:N Quote through quote_likes — both Eloquent directions are defined (Quote::likes and User::likedQuotes).
- Quote 1:N QuoteLike and User 1:N QuoteLike — each like row references one quote and one user.
- Cascade delete is applied to quotes.user_id, quote_likes.quote_id, and quote_likes.user_id.
- The pair (quote_id, user_id) is unique, so the same user cannot like a quote twice.
- Like counts are calculated with an aggregate query; there is no denormalised counter to drift out of sync.
- Quotes are hard-deleted; the application does not use soft deletes, so the feed and counts always reflect the current table.

7.6 Seed Data and Demonstrated Content

The database seeder creates eight users with Arabic names, including the demo account, and eight Arabic cybersecurity quotes. It also assigns 27 likes across the collection. The seed is idempotent: users and quotes are found or created by their unique fields, and likes are attached without duplicates.

- Users: 8 (one demo account and seven community members).
- Quotes: 8, all Arabic cybersecurity lessons, each 47–52 characters long.
- Likes: 27 in total, between 2 and 4 per quote.
- Publication dates: 14 February 2023 to 5 March 2026, spread across eight distinct months.
- Ordering: newest first, so the March 2026 quote appears at the top of the feed.

Table 15. Selected seed quotes (illustrative)

Author	Quote as displayed	English meaning	Date	Likes
خالد العتيبي	كلمة المرور القوية هي خط الدفاع الأول عن بياناتك.	A strong password is the first line of defence for your data.	14 Feb 2023	4
سارة الحربي	فكّر قبل أن تنقر؛ فالتصيّد الإلكتروني يستغل العجلة.	Think before you click; phishing exploits haste.	3 Nov 2023	3
يوسف المطيري	الخصوصية ليست سرية؛ بل سيطرة كاملة على بياناتك.	Privacy is not secrecy; it is full control over your data.	5 Mar 2026	3

The table above shows three of the eight seeded quotes. The Arabic column is rendered right-to-left, exactly as it appears in the application; the English column is provided only as a reference for readers who do not read Arabic.

7.7 Data Retention and Operational Notes

- All application data lives in the SQLite file configured by DB_DATABASE; no uploaded files or object storage are used.
- Deletion is permanent. Removing a user removes that user's quotes and likes; removing a quote removes its likes.
- Timestamps are stored in UTC (the application timezone) and displayed as abbreviated English dates.
- The seeder can be re-run safely, but migrate:fresh --seed is the recommended way to rebuild the demonstration database exactly.
- The seeded database file is included in the repository so the project can be demonstrated without running the seeder.

8. Requirements Traceability

The matrix below connects every functional requirement to the code that implements it and to the check that demonstrates it. Verification was performed against the running application and the seeded SQLite database on 27 October 2026.

Table 16. Functional requirements traceability

ID	Requirement	Implementation	Verification
FR-01	All quotes with author, date, and like count, newest first	QuoteController@index; welcome.blade.php cards and empty state	Guest feed checked in the browser; row order and counts compared with SQLite at five screen widths.
FR-02	Sign-up with name 2–50, unique email, password at least 8, confirmed	AuthController@signup; signup.blade.php; User hashed cast	Valid registration, duplicate email, short password, and mismatch checked; stored value inspected as a hash.
FR-03	Login with email and password, optional remember-me	AuthController@login; login.blade.php; Laravel session guard	Demo login and wrong-password attempts checked; session state confirmed in the header.
FR-04	Logout ends the session	AuthController@logout; header logout form	Logout checked; a protected POST afterwards redirected to login.
FR-05	Publish a quote of 3–155 characters	QuoteController@store; share form in welcome.blade.php	2, 3, 155, and 156 character inputs checked; success and inline errors observed.
FR-06	Edit only own quotes	QuoteController@edit and update; ensureOwner ownership check	Owner edit succeeded; another signed-in user received HTTP 403.
FR-07	Delete only own quotes and cascade likes	QuoteController@destroy; quote_likes foreign key cascade	Owner delete removed the quote and its like rows; non-owner received HTTP 403.
FR-08	Like or unlike with a persistent count	LikeController@toggle; quote_likes table and unique constraint	Toggle checked, page reloaded, count persisted; guest attempt redirected to login.

Non-functional requirements are verified through the implementation evidence in Section 6 and the checklist in Appendix B. Security controls were inspected in the source and the database; responsiveness

was checked at five viewport widths; Arabic rendering was checked on the live feed; and page load was measured locally.

9. Changes from the Original SRS

The attached version 1.2 was written before the final design and implementation were completed. This version keeps the useful structure of that document but corrects and extends it so that it describes the system that was actually delivered.

- Programme and version: the document is now framed for the Summit Bootcamp, Programming and Development Track, and is issued as version 2.0 on 27 October 2026.
- Product positioning: QuoteIt is described as a community vault for Arabic cybersecurity quotes. The earlier text described a generic English-language quote site.
- Database-backed content: quotes, users, and likes are persisted in SQLite. The feed is no longer hardcoded, and the document now describes the schema, migrations, and seeder.
- New feature — Like/Unlike: a new quote_likes table, a toggle route and controller, a filled or unfilled heart state, a persistent count, and a unique user–quote constraint. This satisfies the mandatory new-feature requirement.
- Arabic and RTL: Arabic quote text and author names are detected automatically and displayed right-to-left with IBM Plex Sans Arabic. The earlier document did not describe direction, alignment, or Arabic typography.
- Visual redesign: the final interface uses a dark slate canvas, cyan accents, a shield brand mark, translucent cards, a hero with live statistics, and a one, two, or three-column responsive grid. The earlier document described the earlier light layout.
- Guest experience: guests now see a dedicated login prompt instead of a hidden form, a read-only like count, and a clear invitation to join.
- Owner controls: Edit and Delete appear only on the owner's own cards, and the server returns HTTP 403 for anyone else. The ownership check is documented explicitly.
- Feedback: success alerts, inline validation errors, an empty state, and a native delete confirmation are now documented. The earlier draft mentioned only generic error messages.
- Use case model: the diagram now includes Like/Unlike, shows Log Out as a member action, and clarifies that Owner is a role rather than a third human actor.
- Requirements: the original seven functional requirements became eight, each with actors, preconditions, flows, postconditions, and acceptance criteria. The original five non-functional requirements were expanded to fourteen quality attributes with implementation evidence.
- Accuracy corrections: the quote limit is consistently 155 characters; the login page is described as using placeholders and a demo-account panel rather than per-field hints; and the demo credentials and seeded counts are stated explicitly.

10. Future Enhancements

The following ideas are outside the scope of the final project. They are recorded for future iterations and do not describe current behaviour.

- Search and filtering by keyword, author, or date.
- Categories, tags, and curated collections for different security topics.
- Comments or threaded discussion around a quote.
- Public profiles showing a member's quotes and liked quotes.
- Pagination or infinite scrolling for a larger collection.
- Password reset, email verification, and account settings.
- Reporting, moderation, and a small administration dashboard.
- Bookmarks, shareable quote links, and social preview cards.
- A full Arabic interface with an English/Arabic language switch.
- A read-only JSON API or mobile client built on the same domain model.

11. Appendix A — Setup and Demonstration

11.1 Prerequisites

- PHP 8.3 or newer with the PDO SQLite extension (the verified environment uses PHP 8.5.0).
- Composer 2.
- Node.js and npm for the Vite build.
- A current browser such as Chrome, Edge, Firefox, or Safari.
- The project repository, including composer.json, package.json, and the database folder.

11.2 Installation and Run

From the project root, run the following commands. On Windows, use copy instead of cp and create the SQLite file with New-Item.

```
composer install
copy .env.example .env
php artisan key:generate
New-Item database\database.sqlite -ItemType File
php artisan migrate --seed
npm install
npm run build
php artisan serve
```

Open http://127.0.0.1:8000 in the browser. If the repository already contains a seeded database.sqlite, the file-creation and migration steps can be skipped. To rebuild the demonstration data from scratch, run php artisan migrate:fresh --seed.

For front-end development with hot reload, run npm run dev in a second terminal instead of npm run build.

11.3 Demo Account

Table 17. Demo credentials

Field	Value
Email	**demo@quoteit.com**
Password	**password123**
Display name	خالد العتيبي
Seeded activity	**Four likes on the collection; author of the February 2023 quote.**

The demo password is for local evaluation only. It is hashed in the database like every other password.

11.4 Suggested Five-Minute Demo Flow

1. Open the home page as a guest. Point out the live statistics (8 quotes, 8 voices, 27 likes) and the newest-first collection.
2. Show that Arabic quotes and author names are right-aligned with the Arabic typeface.
3. Try to like a quote. Observe that the guest control is labelled “Log in to like quotes” and that a write attempt redirects to login.
4. Log in with the demo account. The header shows the user's name and a Log out button; the guest prompt becomes the share form.
5. Publish a short Arabic quote. A success alert appears and the new quote is first in the feed.
6. Like the new quote, reload the page, and show that the filled heart and count persist. Unlike it to restore the count.
7. Edit the new quote, save, and show that the text changes while the author and likes remain. Then delete it, confirm the dialog, and show that the feed and totals update.
8. Log out and confirm that the interface returns to the guest view.

11.5 Automated Check

The repository includes a Pest test suite. Running php artisan test executes two tests: a feature test that requests the home page and asserts HTTP 200, and a unit smoke test. Both pass in the verified environment. The remaining functional checks in Appendix B were performed manually because the final project's automated coverage is intentionally minimal.

```
php artisan test
```

Result: 2 tests passed, 2 assertions, in approximately 0.1 seconds.

Appendix B — Verification Checklist

The checklist below records the checks performed against the final build on 27 October 2026. Each item was observed in the running application, in the database, or through the command line.

Table 18. Final verification checklist

#	Check	Method	Result
1	Guest sees all eight quotes in newest-first order with author, date, and like count	Browser and SQLite query	Passed
2	Guest sees the sharing prompt and the read-only like control	Browser	Passed
3	Arabic quotes and author names render right-to-left with the Arabic typeface	Browser	Passed
4	Layout has no horizontal overflow at 320, 390, 768, 1024, and 1440 px	Browser at five widths	Passed
5	A valid sign-up creates an account and signs the user in	Browser	Passed
6	Duplicate email, short password, and mismatched confirmation are rejected	Browser	Passed
7	Demo login succeeds and logout returns to the guest view	Browser	Passed
8	Quote lengths 3 and 155 are accepted; 2 and 156 are rejected	Browser	Passed
9	The owner can edit a quote and a non-owner receives HTTP 403	Browser and HTTP request	Passed
10	Deleting a quote removes its like rows and updates the totals	Browser and SQLite query	Passed
11	Like and unlike persist after a page reload	Browser and SQLite query	Passed
12	A guest cannot add a like	Browser	Passed
13	Passwords are stored as hashes, not plain text	Database inspection	Passed
14	Seed counts are 8 users, 8 quotes, and 27 likes	SQLite query	Passed
15	The Pest test suite passes	php artisan test	Passed

This document describes QuoteIt version 2.0 as implemented for the Summit Bootcamp, Programming and Development Track. Every functional requirement in Section 3 was checked against the running application, and the content of every table matches the final database and interface.

— End of document —